

Architecture Explanation

وثيقة شرح المعمارية التقنية

منصة نَسَق | NASSAQ

(1) الهدف من هذه الوثيقة

هذه الوثيقة تشرح المعمارية التقنية الحالية أو المعمارية المرجعية المطلوبة لمنصة نَسَق | NASSAQ:

- فهم شكل النظام الحالي
- فهم طريقة تنظيم المشروع
- فهم أماكن المنطق المختلفة
- فهم طريقة تدفق البيانات
- معرفة كيفية استكمال المشروع دون كسر البنية
- معرفة أين يجب وضع كل جزء جديد من الكود
- الحفاظ على مبدأ أن النظام حقيقي وديناميكي وليس مجرد واجهات عرض

(2) نوع المعمارية العامة

Overall Architecture Style

نَسَق يجب التعامل معه كالتالي:

النظام هو تطبيق ويب متكامل Full-Stack مبني على نموذج:

Frontend + Backend + Database

وقد يكون تنفيذه الحالي واحدًا من حالتين:

الحالة الأولى: Monolith Application

في هذه الحالة يكون:

- الفرونت إند
- الباك إند
- APIs
- وربما حتى الاتصال بقاعدة البيانات

كلها داخل مشروع واحد أو مستودع واحد، مع تنظيم داخلي للفصل بين الطبقات.

الحالة الثانية: Frontend / Backend Separated

في هذه الحالة يكون هناك:

- مشروع Frontend مستقل
- مشروع Backend مستقل
- قاعدة بيانات مستقلة
- تواصل بين الطرفين عبر REST APIs أو ما شابه

التوصية المرجعية للـ Agent

حتى لو كان المشروع الحالي منفذًا بشكل Monolith، يجب على الـ Agent أن يفهمه عمليًا على أنه يتكون من 3 طبقات منطقية أساسية:

1. **Presentation Layer**
الواجهات، الصفحات، المكونات، والـ state
2. **Application / Business Logic Layer**
الخدمات، القواعد، التحقق، الصلاحيات، منطق التشغيل
3. **Data Layer**
قاعدة البيانات، النماذج، العلاقات، الاستعلامات، الـ persistence

بمعنى آخر:

حتى لو كان الكود داخل مشروع واحد، لا يجب خلط الطبقات أو وضع منطق العمل داخل الواجهة.

(3) الـ Stack المستخدم أو المتوقع

Technology Stack

بسبب طبيعة المشروع، يجب أن يفهم الـ Agent أن نَسَق مبني أو يجب أن يُبنى باستخدام Stack حديث يدعم:

- تعدد الأدوار
- تعدد المدارس
- CRUD ثقيل
- جداول ديناميكية
- صلاحيات
- تقارير
- لوحات بيانات
- APIs
- قابلية التوسع

Frontend Stack

غالبًا أو مرجعيًا:

- **React** أو إطار مشابه
- React Router أو أي نظام Routing مشابه
- Component-based UI Architecture
- State Management محلي + مركزي عند الحاجة
- API Client Layer لفصل الواجهة عن الـ backend

Backend Stack

غالبًا أو مرجعيًا:

- **Node.js**
- REST API Architecture
- Services Layer
- Auth / RBAC Middleware
- Tenant Enforcement Middleware
- Audit Logging Layer

Database

غالبًا أو مرجعيًا:

- قاعدة بيانات علائقية مثل:
 - PostgreSQL
 - MySQL
 - أو ما يشبهها

لأن النظام يعتمد على:

- علاقات معقدة
- جداول مترابطة
- صلاحيات
- سجلات
- تقارير
- سياقات متعددة

4) هيكل المشروع

Project Structure

يجب أن تفهم المشروع يجب أن يكون منظمًا منطقيًا على الأقل بهذا الشكل:

إذا كان المشروع Frontend + Backend منفصلًا

Frontend

- src/pages
- src/components
- src/layouts
- src/routes
- src/hooks
- src/state أو src/store
- src/api أو src/services

- src/utils
- src/constants

Backend

- server
- routes
- controllers
- services
- models
- repositories أو data access layer
- middleware
- auth
- validators
- logs
- config

إذا كان المشروع Monolith

يجب رغم ذلك أن يكون منظمًا داخليًا إلى:

- قسم frontend
- قسم backend
- قسم database/config
- قسم shared logic إن لزم

ولا يجوز خلط:

- مكونات الواجهة
 - مع منطق قواعد العمل
 - مع استعلامات قاعدة البيانات
 - مع الصلاحيات
- داخل نفس الملفات بطريقة فوضوية

5) أين توجد الـ Routes؟

Routing Structure

Frontend Routes

الـ routes الخاصة بالواجهة يجب أن تكون مسؤولة فقط عن:

- ربط URL بالصفحة المناسبة
- تحميل Layout مناسب
- التحقق من وجود جلسة أو صلاحية مبدئية
- تمرير المستخدم إلى الصفحة أو redirect

- platform/dashboard/
- platform/schools/
- platform/users/
- school/dashboard/
- school/users-classes/
- school/schedule/
- teacher/dashboard/

ما الذي لا يجب أن يوجد داخل route definition؟

- منطق عمل ثقيل
- استعلامات مباشرة
- قواعد صلاحيات معقدة مكتوبة عشوائيًا
- قرارات Tenant معقدة داخل كل route يدويًا

Backend Routes / API Routes

الـ backend routes يجب أن تكون endpoints واضحة ومسؤولة عن استقبال الطلبات فقط، مثل:

- api/platform/schools/
- api/platform/users/
- api/school/students/
- api/school/teachers/
- api/school/classes/
- api/school/schedule/
- api/attendance/
- api/assessments/
- api/notifications/
- api/reports/

وظيفة الـ route backend

- استقبال الطلب
- تمريره إلى controller أو service مناسب
- تطبيق middleware
- إرجاع response منظم

(6) أين توجد الـ Models؟

Data Models

الـ models يجب أن تمثل الكيانات الحقيقية داخل النظام، مثل:

- UserIdentity

- User
- Role
- School
- Student
- Teacher
- Parent
- Class
- Subject
- Schedule
- AttendanceRecord
- Assessment
- Notification
- AuditLog
- RegistrationRequest
- SchoolSettings

وظيفة ال models

- تمثيل بنية الجداول
- تعريف العلاقات
- دعم عمليات القراءة والكتابة
- توحيد الوصول إلى البيانات

قاعدة مهمة

ال model لا يجب أن يحتوي على منطق أعمال ثقيل جدًا. المنطق المعقد يجب أن يكون في **services layer**، وليس داخل model بشكل عشوائي.

(7) أين توجد ال Services؟

Services Layer

هذه من أهم الطبقات في نَسَق.

ال Services هي الطبقة التي يجب أن تحتوي على **منطق الأعمال الحقيقي**، مثل:

- إنشاء مدرسة جديدة
- إنشاء طالب وربطه بولي أمر
- توزيع طالب على فصل
- إسناد معلم لمادة
- توليد جدول
- تسجيل حضور
- احتساب نسبة حضور
- إنشاء إشعار
- فرض صلاحية
- تحديث العدادات

- تسجيل audit log

أمثلة لخدمات متوقعة

- schoolService
- userService
- studentService
- teacherService
- scheduleService
- attendanceService
- assessmentService
- notificationService
- reportingService
- authService
- tenantService
- auditLogService

قاعدة أساسية

إذا احتاجت أي شاشة إلى منطق حقيقي، فيجب أن يمر عبر **service layer**، وليس أن يُكتب داخل:

- component
- route
- controller
- أو query مباشر في كل مكان

8) أين توجد الـ APIs؟

API Layer

الـ APIs في نَسَق هي الجسر بين الواجهة وقاعدة البيانات عبر الباك إند.

دور الـ API

- استقبال أوامر الواجهة
- تطبيق التحقق
- تطبيق الصلاحيات
- تطبيق عزل المدرسة / tenant
- استدعاء service مناسب
- إرجاع بيانات حقيقية

أنواع الـ APIs المتوقعة

CRUD APIs

مثل:

- إنشاء مدرسة
- تعديل مستخدم
- حذف فصل
- تحديث جدول

Operational APIs

مثل:

- توليد الجدول
- تسجيل الحضور
- إرسال إشعار
- نشر نسخة من الشروط والأحكام

Reporting APIs

مثل:

- نسب الحضور
- عدد المدارس
- عدد المعلمين المستقلين
- مؤشرات dashboard
- تقارير شهرية

System APIs

مثل:

- login
- logout
- switch role
- active sessions
- force password change

قاعدة مهمة

أي بيانات تظهر في الواجهة يجب أن تأتي من API حقيقي أو Query حقيقي في backend. لا يجب أن تبني cards أو charts على arrays ثابتة داخل الواجهة.

(9) أين يوجد State Management؟

State Management

في نَسَق، ال state management يجب أن يكون مقسمًا إلى 3 أنواع:

1. Local UI State

خاص بالمكون نفسه، مثل:

- فتح وغلق modal
- قيمة input
- active tab
- pagination الحالية
- sort direction

هذا النوع يبقى داخل component أو hook محلي.

2. Page State / Feature State

خاص بالصفحة أو الوحدة، مثل:

- قائمة الطلاب
- filters
- selected class
- selected date range
- loading state
- current report data

هذا يمكن أن يُدار عبر:

- custom hooks
- feature store
- أو state management مركزي إذا كان مناسبًا

3. Global App State

وهذا يجب أن يشمل فقط الأشياء المشتركة فعليًا، مثل:

- المستخدم الحالي
- الدور النشط
- tenant context
- session info
- global settings
- language
- permissions summary
- notification count العامة

قاعدة مهمة

لا يجب استخدام global state كبديل عن قاعدة البيانات.
الـ global state فقط يساعد على:

- تحسين UX
- تقليل الطلبات المتكررة
- حفظ سياق الجلسة

لكنه ليس مصدر الحقيقة.
المصدر الحقيقي دائماً هو backend + database.

10) كيف يتم التعامل مع Authentication؟

Auth Flow

المصادقة في نَسَق يجب أن تعتمد على نظام واضح وآمن.

السيناريو الأساسي

1. المستخدم يسجل الدخول
2. backend يتحقق من:
 - البريد / الهاتف
 - كلمة المرور
 - حالة الحساب
 - هل الحساب مقفول أو معلق
3. يتم إنشاء session أو token
4. يتم تحميل:
 - user identity
 - roles
 - active role
 - tenant context إن وجد
5. يتم توجيه المستخدم إلى dashboard المناسب

بعد تسجيل الدخول

يجب أن يعرف النظام:

- من هو المستخدم؟
- ما هي هويته الموحدة؟
- ما هي أدواره؟
- ما هو دوره النشط الآن؟
- ما هو tenant / school المرتبط به؟
- ما هي صلاحياته الفعلية؟

11) كيف يتم التعامل مع Authorization / Role

Permissions؟

Role-Based Access Control

نَسَق يجب أن يعمل وفق RBAC + Context-Based Permissions

أي:

- هناك Role أساسي
- وهناك Context مرتبط به
- وهناك Permissions فعلية ناتجة عن الاثنين

أمثلة

- Platform Admin يرى كل المدارس
- School Admin يرى مدرسة واحدة فقط
- Teacher يرى فصوله فقط
- Parent يرى أبنائه فقط

أين تطبق الصلاحيات؟

1. على مستوى الواجهة

لإخفاء أو إظهار:

- الصفحات
- الأزرار
- العمليات
- التبويبات

2. على مستوى الـ backend

وهذا هو الأهم.

حتى لو حاول المستخدم تجاوز الواجهة، يجب على backend أن يمنع:

- الوصول غير المصرح
- تنفيذ عمليات لا يملك صلاحيتها
- قراءة بيانات tenant آخر

قاعدة أساسية

الواجهة ليست طبقة أمان.

الأمان الحقيقي يجب أن يطبق في backend.

12) كيف يتم Tenant Isolation؟

Multi-Tenant Isolation

هذه من أخطر النقاط وأهمها.

كل عملية في النظام يجب أن تحترم **Tenant Context**.

كيف يحدث ذلك؟

عندما يكون المستخدم مرتبطًا بمدرسة:

- يتم تحميل tenant_id أو school_id في الجلسة أو السياق
- كل query أو service يجب أن تستخدم هذا السياق
- كل API يجب أن يتحقق أن البيانات المطلوبة تخص نفس المدرسة

أمثلة

School Admin

لا يجب أن يرى:

- طلاب مدرسة أخرى
- فصول مدرسة أخرى
- تقارير مدرسة أخرى

Teacher

لا يجب أن يرى إلا:

- المدرسة التابعة له
- فصوله
- مواد
- طلابه حسب التعيين

أين يطبق Tenant Isolation؟

- في middleware
- في services
- في database queries
- في reporting queries
- في dashboards
- في exports
- في search

قاعدة صارمة

أي endpoint أو query أو report أو chart لا يحترم tenant context يعتبر خطأ معماريًا.

13) كيف يتم التعامل مع Audit Logs؟

Audit Logging

لأن نَسَق نظام تشغيلي حقيقي، فلا بد من وجود سجل تدقيق مركزي.

ما الذي يجب تسجيله؟

العمليات الحساسة مثل:

- إنشاء / تعديل / حذف مدرسة
- إنشاء / تعديل / تعليق مستخدم
- تغيير صلاحيات
- نقل طالب
- تعديل جدول
- تسجيل حضور
- فرض تغيير كلمة مرور
- قفل / فتح حساب
- نشر شروط وأحكام
- تغيير إعداد عام
- إنهاء الجلسات

ما الذي يجب أن يحتويه الـ log؟

- نوع العملية
- المستخدم الذي نفذها
- التاريخ
- الوقت
- الـ role المستخدم وقت التنفيذ
- الـ tenant / المدرسة إن وجدت
- الكيان المتأثر
- القيمة القديمة والجديدة إذا أمكن
- عنوان IP أو معلومات الجهاز إن كانت متاحة

أين يتم تسجيله؟

في backend عبر:

- auditLogService
- middleware
- أو event hooks منظمة

قاعدة مهمة

لا يجب أن يعتمد audit logging على الفرونت إند فقط.
الـ backend هو الذي يجب أن يضمن تسجيل العملية.

14) كيف يتم التعامل مع Charts و Reports؟

Reporting & Analytics Architecture

الرسوم البيانية والتقارير في نَسَق ليست عناصر تجميلية.
هي يجب أن تُبنى من بيانات فعلية.

ما هو المبدأ؟

كل:

- chart
- KPI
- counter
- dashboard card
- attendance rate
- teacher ratio
- student count
- pending requests
- AI-enabled schools

يجب أن يأتي من:

- database query
- aggregate query
- reporting service
- أو materialized/statistical layer إن لزم

ما الذي يمنع؟

- arrays ثابتة داخل الواجهة
- بيانات demo في chart
- أرقام مكتوبة يدويًا
- placeholder percentages
- fake analytics

كيف يجب بناؤها؟

1. Reporting Service

طبقة مخصصة لحساب:

- totals
- ratios
- trends
- grouped results
- time-based aggregation

2. Reporting APIs

تُرجع:

- card metrics
- chart datasets
- filters-aware results
- tenant-aware reports

3. Frontend Visualization Layer

تعرض البيانات فقط، ولا تختبرها.

مثال

كارت:

نسبة حضور الطلاب

يجب أن:

- يقرأ عدد سجلات الحضور الفعلية
- يقارن بين الحضور والغياب
- يحسب النسبة الحقيقية
- يعرضها للمستخدم
- ويعيد التحديث عند تغير البيانات

15) كيف تتدفق البيانات داخل النظام؟

End-to-End Data Flow

ال Agent يجب أن يفهم أن التدفق الطبيعي للبيانات في نَسَق هو:

مثال: إنشاء طالب

1. المستخدم يفتح صفحة إدارة المستخدمين والفصول
2. يضغط "إضافة طالب"
3. يدخل البيانات
4. الفرونت إند ينفذ validation أولي
5. يرسل الطلب إلى API
6. backend يتحقق من:
 - الصلاحية
 - tenant context
 - uniqueness
 - ولي الأمر
 - الفصل
7. service تنشئ السجلات اللازمة
8. يتم الحفظ في قاعدة البيانات
9. يتم تسجيل العملية في audit log
10. ترجع الاستجابة للواجهة
11. تتحدث:

- قائمة الطلاب
- عدد الطلاب
- بيانات الفصل
- تقارير المدرسة إن كانت مرتبطة

هذا هو النمط المطلوب في كل العمليات.

16) قواعد معمارية صارمة يجب أن يفهمها الـ Agent

ممنوع

- كتابة business logic ثقيل داخل components
- حفظ البيانات محليًا فقط وإظهار نجاح مزيف
- استخدام mock data بدل backend
- كسر tenant isolation
- تكرار logic الصلاحيات في كل صفحة بشكل عشوائي
- بناء التقارير من بيانات frontend
- تنفيذ charts بشكل تجميلي فقط
- خلط model logic مع UI logic
- الاعتماد على static fallback لإخفاء فشل الربط

مطلوب

- فصل واضح بين الطبقات
- service layer حقيقية
- API layer نظيفة
- RBAC حقيقي
- tenant context واضح
- reporting layer مبنية على DB
- audit logs مركزية
- source of truth = database

17) الخلاصة التنفيذية للـ Agent

يجب أن يفهم الـ Agent أن معمارية نَسَق تقوم على:

- **Frontend** مسؤول عن العرض والتفاعل و UI state
- **Backend** مسؤول عن business logic والصلاحيات والتحقق والتقارير
- **Database** هي المصدر الوحيد للحقيقة
- **Tenant Context** هو الأساس في عزل المدارس
- **Unified User Identity** هو الأساس في إدارة المستخدمين والأدوار
- **Services Layer** هي مركز منطق التشغيل
- **Reporting Layer** هي المسؤولة عن dashboards و charts و KPIs
- **Audit Logging** جزء أساسي من العمليات الحساسة
- **No Fake Save / No Mock Final Data / No Hardcoded Metrics**